



IONIAN UNIVERSITY

Department of Informatics

An Overview of the Convex Hull Problem and Its Practical Applications

Student: Mohammad Matin Marzie

AM: inf2022001

Supervisor: Theodore Andronikos

Corfu

February 10, 2026

Contents

1	Abstract	2
2	Introduction & Problem Statement	3
2.1	Computational Geometry	3
2.2	What is a Convex Hull (2D)?	3
2.3	In 3 Dimensions	5
2.4	In High Dimensions	5
3	Solutions and Algorithms in 2D	6
3.1	brute force naive algorithm	6
3.1.1	Pseudocode & Explanation	6
3.1.2	Time Complexity	7
3.1.3	python implementation	8
3.2	Graham Scan	9
3.2.1	Pseudocode & Explanation	12
3.2.2	Time Complexity	13
3.2.3	python implementation	13
3.3	Jarvis March (Gift Wrapping)	14
3.3.1	Pseudocode Explanation	15
3.3.2	Time Complexity	16
3.3.3	python implementation	17
3.4	Optimal Output-Sensitive Convex Hull Algorithms	18
4	Convex hull in practice	19
4.1	Convex hull in Statistics	19
4.2	Convex hull in Computer Graphics	19
4.3	Convex hull in Machine Learning Fuzzy clustering	19
5	Bibliography	21

1 Abstract

The convex hull problem represents one of the most fundamental challenges in computational geometry, serving as a critical building block for a wide array of algorithms in multidimensional space. This survey provides a comprehensive overview of the convex hull problem, beginning with its formal mathematical definitions and geometric interpretations in 2D, 3D, and higher-dimensional Euclidean spaces. We analyze the evolution of algorithmic solutions, ranging from the foundational $O(n^3)$ brute-force approach to the $O(n \log n)$ Graham Scan and the output-sensitive Jarvis March. Particular attention is given to Chan's algorithm, which achieves optimal $O(n \log h)$ complexity by hybridizing pre-processing and gift-wrapping techniques. Beyond theoretical analysis, this survey explores the transformative utility of the convex hull in applied sciences. In the realm of robust statistics, we examine how convex hull peeling and area sensitivity offer non-parametric alternatives for multivariate location estimation and outlier detection in astronomical datasets. In computer graphics, we discuss the role of convex hulls and Approximate Convex Decomposition (ACD) in optimizing real-time collision detection via the Gilbert-Johnson-Keerthi (GJK) algorithm. Finally, we review recent integrations in machine learning, specifically focusing on how convex hulls enable fuzzy clustering models to define irregular data partitions. By bridging the gap between abstract geometric theory and modern computational needs, this work highlights the enduring relevance of the convex hull in solving complex, high-dimensional problems.

2 Introduction & Problem Statement

2.1 Computational Geometry

Geometry is a fundamental branch of mathematics concerned with the study of space and the properties of figures, such as shape, size, distance, and relative position [11]. Originating from early practical needs in measurement and construction in ancient Egypt [10], as well as astronomical observations in Mesopotamia [5], geometry gradually evolved into a rigorous theoretical discipline based on axioms, definitions, and formal proofs. Classical Euclidean geometry introduced core concepts such as points, lines, planes, angles, and polygons, which remain central to mathematical education and scientific reasoning [15]. Since the 19th century, geometry has expanded beyond Euclidean space to include non-Euclidean, projective, affine, and higher-dimensional geometries, significantly broadening its scope [16].

Today, geometry plays a crucial role across mathematics and the natural sciences. It underpins fields such as physics, engineering, computer graphics, architecture, and data analysis, providing a mathematical framework for describing complex structures including manifolds, surfaces, and high-dimensional spaces. Modern geometry is divided into numerous subfields—such as differential, algebraic, discrete, and **convex geometry**—each emphasizing different methods and properties [16]. As geometric problems have become more complex and data-driven, particularly in higher dimensions, the need for efficient algorithmic solutions has grown increasingly important.

Computational geometry is the branch of computer science that studies algorithms for solving geometric problems [4].

Within computational geometry, a large class of problems focuses on finite sets of points in Euclidean space. Typical tasks include determining spatial relationships, detecting intersections, computing proximity structures, and identifying boundary representations of point sets. Among these problems, the convex hull occupies a central role, as it provides a compact geometric summary of a dataset by describing its outermost structure. Moreover, many geometric algorithms—such as collision detection, clustering, and shape analysis—either rely on convex hull computations or use them as a preprocessing step, as will be discussed in Section 4. But what exactly is meant by a convex hull, and how is it formally defined?

2.2 What is a Convex Hull (2D)?

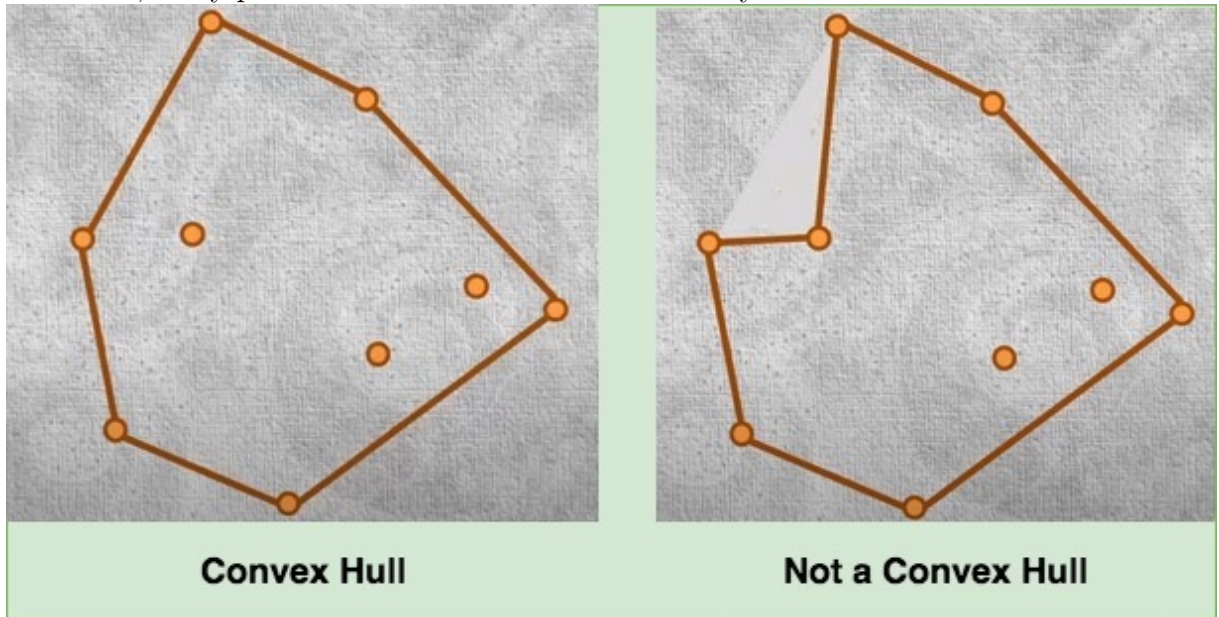
As discussed in the previous subsection, the input objects of algorithms in this survey are represented as sets of points in 2D, for example, $X = \{p_1, p_2, p_3, \dots\}$ in the Euclidean plane $\mathbb{E}^2$, where each point p_i is a pair $p_i = (x_i, y_i)$ with $x_i, y_i \in \mathbb{R}$.

A set of points Q , in a Euclidean space is called **convex** if it contains the line segment

connecting every pair of its points. The **convex hull** of a given set X can be defined in several equivalent ways [14]:

1. The (unique) minimal convex set containing X .
2. The intersection of all convex sets containing X .
3. The set of all convex combinations of points in X .
4. The union of all simplices with vertices in X .

By definition, every point in X lies either on the boundary of the convex hull or inside it.



Example of Convex Hull from [13]

An Intuitive Interpretation

To give an intuitive geometric interpretation, imagine a set of nails placed on a flat board representing the points in $X \subset \mathbb{E}^2$. If we stretch a rubber band (or rope) around all the nails and then release it so that it tightly encloses them, the shape formed by the stretched rubber band corresponds to the convex hull of the set X . In other words, the convex hull is the smallest convex polygon that contains all the points, just as the rubber band wraps around the outermost nails.

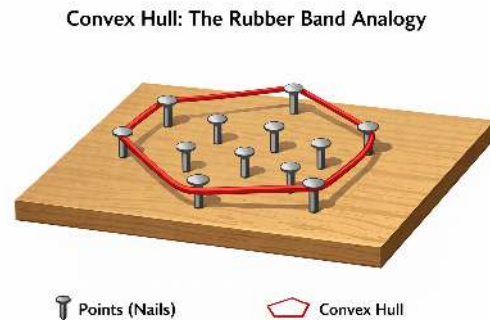


Figure 1: Convex Hull: The Rubber Band Analogy(Generated by ChatGpt)

2.3 In 3 Dimensions

Extending the concept of convex hulls to three-dimensional space, the input is represented as a set of points

$X = \{p_1, p_2, \dots, p_n\} \subset \mathbb{E}^3$, where each point p_i is a vector $p_i = (x_i, y_i, z_i) \in \mathbb{R}^3$. The convex hull in 3D is defined analogously: it is the smallest convex polyhedron that contains all points in X . Geometrically, one can imagine stretching a flexible sheet or wrapping a tight elastic surface around the outermost points.

Like in 2D, the convex hull can be defined equivalently as the minimal convex set containing X , the intersection of all convex sets containing X , the set of all convex combinations of points in X , or the union of all simplices with vertices in X . In 3D, simplices are tetrahedra formed from four points. Convex hulls in 3D also introduce additional computational challenges, such as detecting coplanar points and managing faces, edges, and vertices in polyhedral structures. The vector representation of points facilitates the computation of dot products and cross products, which are fundamental in determining orientations and face normals during algorithmic construction.

A 3D convex hull can be visualized as the minimal polyhedral surface enclosing a cloud of points. The geometric intuition extends naturally from 2D: each face of the polyhedron is flat, and each edge connects vertices on the boundary. Convex hull computations in 3D are widely used in collision detection (computer graphics 4), 3D modeling, and mesh generation.

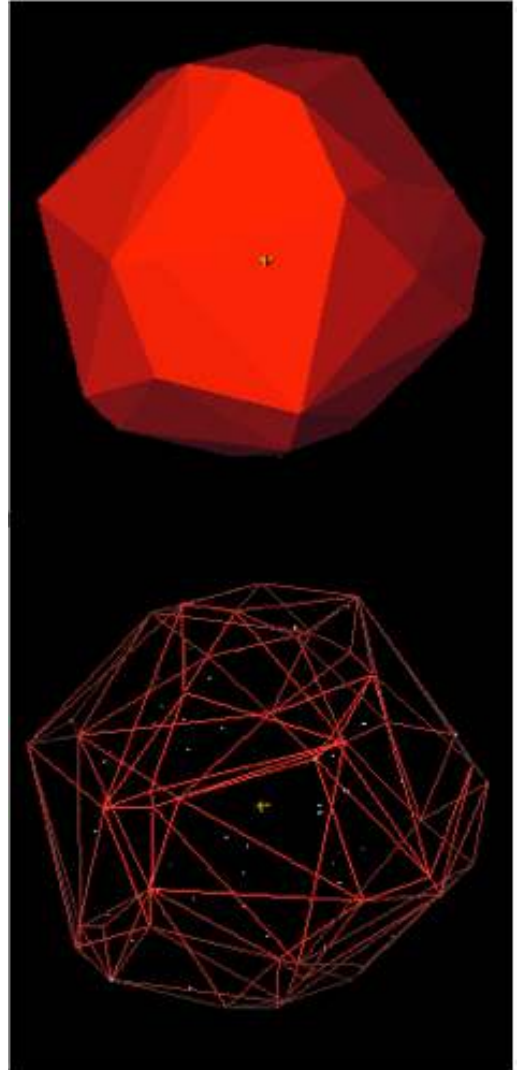


Figure 2: 3D convex hull of 120 point cloud from [14]

2.4 In High Dimensions

When moving to higher-dimensional spaces, $\mathbb{E}^d$ with $d > 3$, the concept of convex hulls generalizes naturally but becomes more abstract. Points are represented as d -dimensional vectors $p_i = (x_1, x_2, \dots, x_d) \in \mathbb{R}^d$, and the convex hull is still defined as the smallest convex set containing X . In these spaces, convex hulls can also be studied in non-Euclidean geometries, where distances and angles may follow different rules, extending the applicability

3 Solutions and Algorithms in 2D

In this section, we present classical algorithms for computing the convex hull in two dimensions. For each algorithm, we describe the main idea, provide pseudocode, analyze the time complexity, and give an implementation in the Python programming language.

We begin with the **brute-force (naive) approach** 3.1, which has a time complexity of $O(n^3)$. We then introduce more efficient algorithms, including **Graham Scan** 3.2 with time complexity $O(n \log n)$ and **Jarvis March** 3.3 with time complexity $O(nh)$, where h denotes the number of points on the convex hull. Finally, we discuss **Chan's algorithm** 3.4, an output-sensitive method that combines ideas from Graham Scan and Jarvis March and achieves a time complexity of $O(n \log h)$.

3.1 brute force naive algorithm

The brute force approach to the convex hull problem is based on the geometric definition of a supporting line. A line segment connecting two points p and q is an edge of the convex hull if and only if all other points in the set lie on the same side of the infinite line passing through p and q as depicted in Figure 3. By testing every possible pair of points, we can identify all boundary segments.

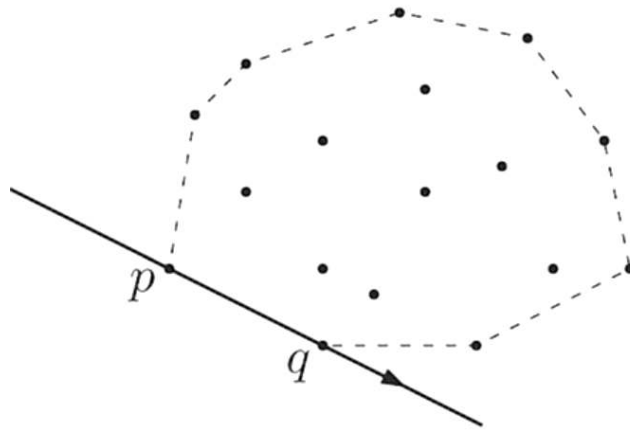


Figure 3: Convex Hull boundary line segment of point p and q

3.1.1 Pseudocode & Explanation

The algorithm iterates through every directed pair of points in counter-clockwise to verify if they form a boundary. The "sidedness" is determined by the **Orientation Test**. For any three points p, q , and r , we calculate the cross product:

$$(x_q - x_p)(y_r - y_p) - (y_q - y_p)(x_r - x_p)$$

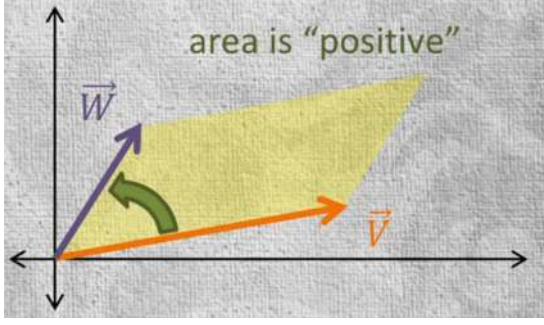


Figure 4: Positive cross product: r is to the left of $\vec{pq}$. Screenshot from [3]

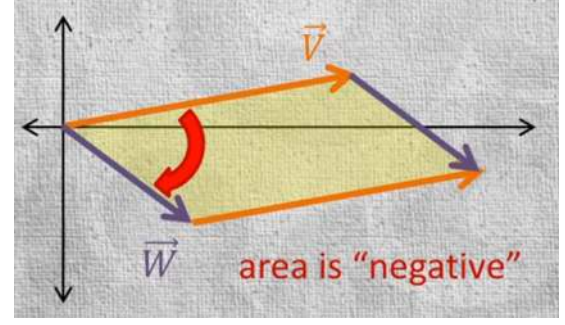


Figure 5: Negative cross product: r is to the right of $\vec{pq}$. Screenshot from [3]

A negative result indicates that point r lies to the right of the directed line $\vec{pq}$ as depicted in Figure 5. If no such point r exists for a pair (p, q) , the segment is part of the hull. Finally, we sort the unique vertices of E in counter-clockwise and return it as a list named L .

Algorithm 1 Brute Force Convex Hull

BruteForceConvexHullX

```

1: Input: A set  $X$  of  $n$  distinct points.
2: Output: A list  $L$  of vertices in counter-clockwise order.
3:  $E \leftarrow \emptyset$  {Initialize set of boundary edges}
4: for all ordered pairs  $(p, q)$  in  $X \times X$  with  $p \neq q$  do
5:    $valid \leftarrow \text{true}$ 
6:   for all points  $r$  in  $X \setminus \{p, q\}$  do
7:     if  $r$  lies to the right of the directed line from  $p$  to  $q$  then
8:        $valid \leftarrow \text{false}$ 
9:     break
10:  if  $valid$  then
11:    Add directed edge  $\vec{pq}$  to  $E$ 
12:  $L \leftarrow$  Sort unique vertices of  $E$  in counter-clockwise order
13: return  $L$ 

```

3.1.2 Time Complexity

To determine the total time complexity $T(n)$, we assign a cost to each step of the algorithm based on the number of times it is executed relative to the input size n .

1. Initialization (Line 4): Constant time cost, c_1 .
2. Outer Loops (Lines 5-6): We select all ordered pairs (p, q) . There are n choices for p and $(n - 1)$ choices for q . This gives $n(n - 1)$ iterations.

3. Inner Loop (Lines 8-12): For every pair (p, q) , we iterate through all other points r . There are $(n - 2)$ such points.
4. Orientation Test (Line 9-10): The cross product calculation and the conditional check are constant time operations, c_2 .
5. Post-processing (Line 15): Sorting the h points found on the hull (where $h \leq n$) takes $O(h \log h)$.

The total cost function is:

$$T(n) = c_1 + \sum_{p=1}^n \sum_{q=1, q \neq p}^n \left(\sum_{r=1, r \neq p, q}^n c_2 \right) + O(h \log h)$$

Approximating the summations:

$$T(n) = c_1 + [n \cdot (n - 1) \cdot (n - 2)] \cdot c_2 + O(n \log n)$$

Expanding the cubic term:

$$T(n) = c_2(n^3 - 3n^2 + 2n) + O(n \log n)$$

In Big O notation, we drop lower-order terms and constant coefficients. Therefore, the final complexity is:

$$O(n^3)$$

3.1.3 python implementation

This implementation follows the pseudocode logic, using a cross-product function to determine orientation and a polar angle sort to return the final list in counter-clockwise order.

```

1
2 import math
3
4 def get_orientation(p, q, r):
5     """
6     Returns the orientation of triplet (p, q, r).
7     Positive: Left turn (Counter-clockwise)
8     Negative: Right turn (Clockwise)
9     Zero: Collinear
10    """
11    return (q[0] - p[0]) * (r[1] - p[1]) - (q[1] - p[1]) * (r[0] - p[0])
12
13 def brute_force_convex_hull(points):
```

```

14     n = len(points)
15     if n < 3:
16         return sorted(points)
17
18     E = [] # Set of directed edges
19
20     # Check every ordered pair (p, q)
21     for i in range(n):
22         for j in range(n):
23             if i == j: continue
24
25             p, q = points[i], points[j]
26             valid = True
27
28             # Check if all other points r lie to the left of directed line pq
29             for k in range(n):
30                 if k == i or k == j: continue
31
32                 r = points[k]
33                 if get_orientation(p, q, r) < 0:
34                     valid = False
35                     break
36
37             if valid:
38                 E.append((p, q))
39
40     # Extract unique vertices from the valid edges
41     hull_vertices = list(set([p for edge in E for p in edge]))
42
43     # Sort vertices CCW to satisfy the Output requirement
44     # We sort by the angle relative to the centroid of the hull points
45     cx = sum(p[0] for p in hull_vertices) / len(hull_vertices)
46     cy = sum(p[1] for p in hull_vertices) / len(hull_vertices)
47
48     hull_vertices.sort(key=lambda p: math.atan2(p[1] - cy, p[0] - cx))
49
50     return hull_vertices
51
52 # Example usage:
53 # pts = [(0, 3), (2, 2), (1, 1), (2, 1), (3, 0), (0, 0), (3, 3)]
54 # print(brute_force_convex_hull(pts))

```

3.2 Graham Scan

The Graham Scan, proposed by Ronald Graham in 1972 [6], is a more efficient method for computing the convex hull. Unlike the brute-force approach that checks every possible pair of points, Graham Scan uses a **sorting mechanism** and a **stack-based** refinement

process. The algorithm first identifies the point with the lowest y -coordinate (the "pivot") among the input set (Figure 6), as shown in Figure 7.

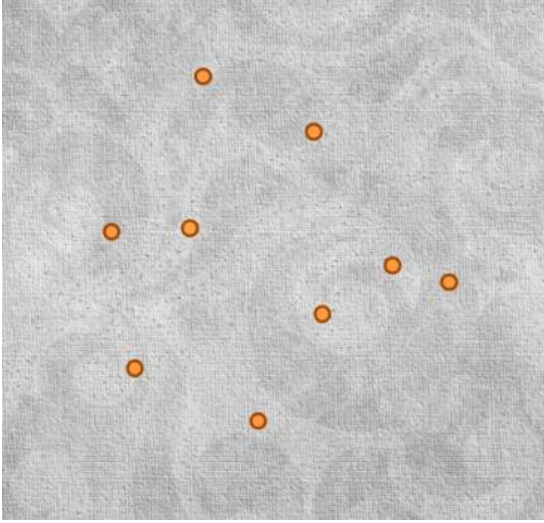


Figure 6: The set of input points. Screenshot from [3]

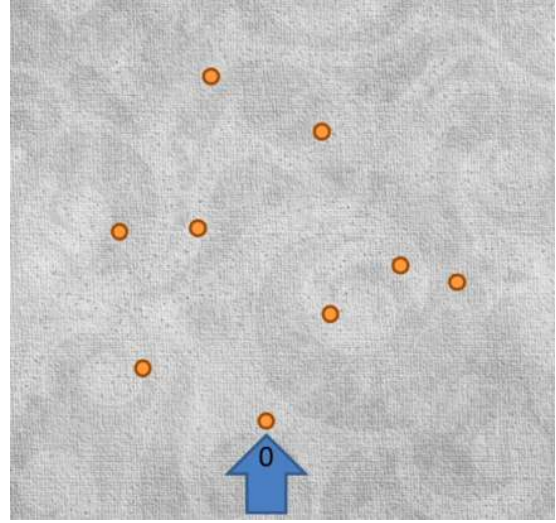


Figure 7: Identifying the pivot (lowest y -coordinate). Screenshot from [3]

The remaining points are sorted based on the polar angle θ they form with this pivot, as depicted in Figure 8. The algorithm then processes the points in this sorted order, maintaining a stack of vertices that form a tentative convex chain. Initially, the pivot p_0 and the first sorted point p_1 are pushed onto the stack (Figure 9).

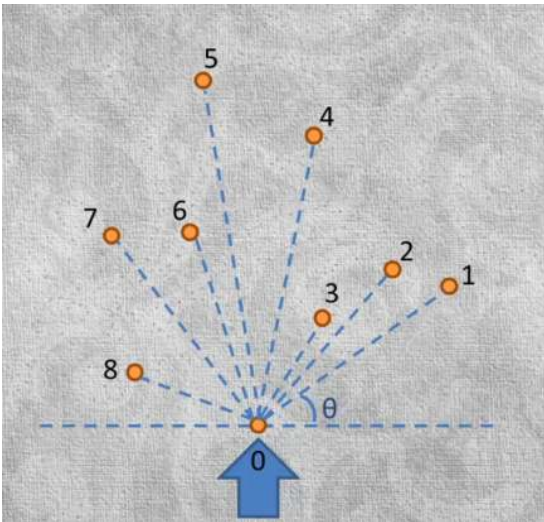


Figure 8: Points sorted by polar angle θ . Screenshot from [3]

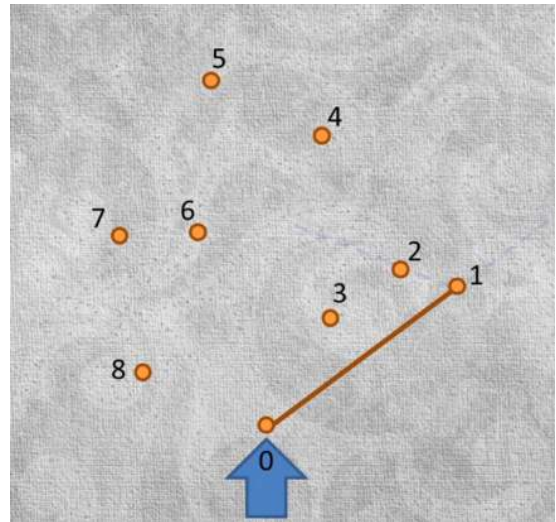


Figure 9: Initially, p_0 and p_1 are added to the stack. Screenshot from [3]

Upon reading each subsequent point p_i , we perform an orientation test. For point p_2 , we calculate the cross-product of $p_0\vec{p}_1$ and $p_1\vec{p}_2$. A positive sign indicates a counter-

clockwise turn; in this case, p_2 is added to the stack (Figure 10). This continues until we find a point that breaks convexity (Figure 11).

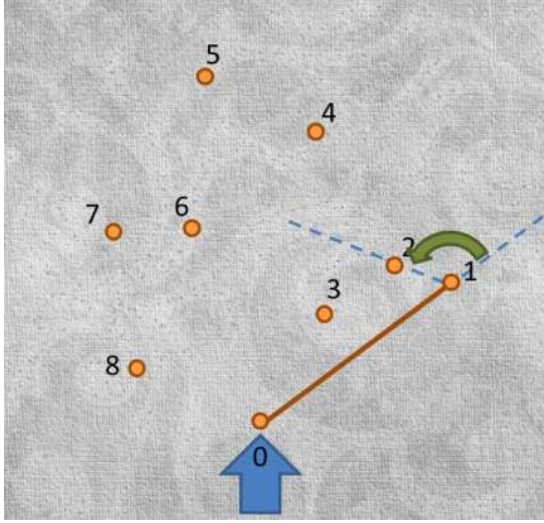


Figure 10: Orientation test for p_2 . Screenshot from [3]

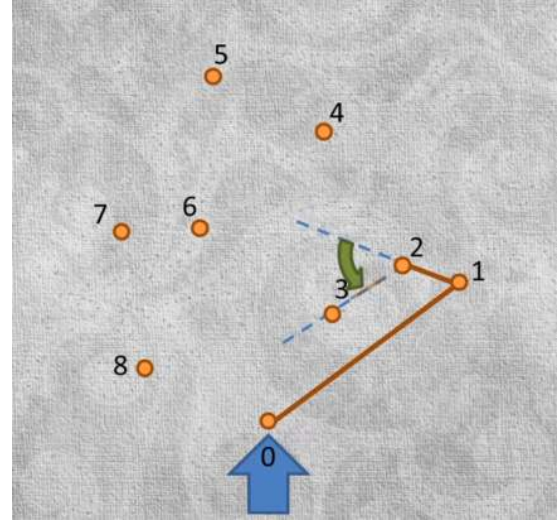


Figure 11: p_3 added after a successful orientation test. Screenshot from [3]

When a clockwise turn (negative cross-product) is encountered, the algorithm must backtrack. In Figure 12, point q_4 creates a right turn relative to $p_2\vec{p}_3$. Consequently, p_3 is popped from the stack. The check continues: if q_4 still creates a right turn relative to the new top of the stack (Figure 13), we pop p_2 as well. This "popping" continues until a counter-clockwise turn is restored (Figure 14), at which point p_4 is finally pushed onto the stack (Figure 15).

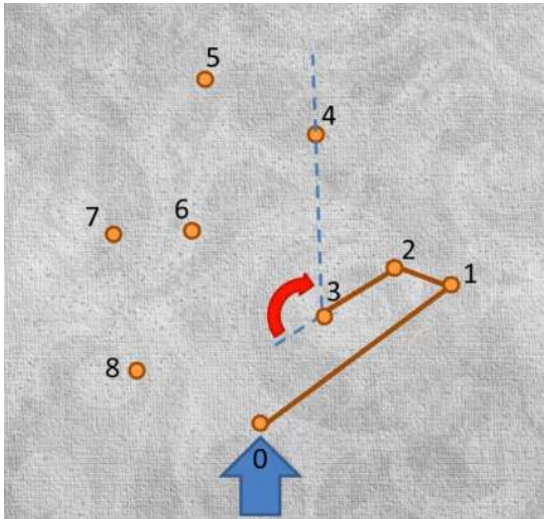


Figure 12: Right turn detected: p_3 is removed. Screenshot from [3]

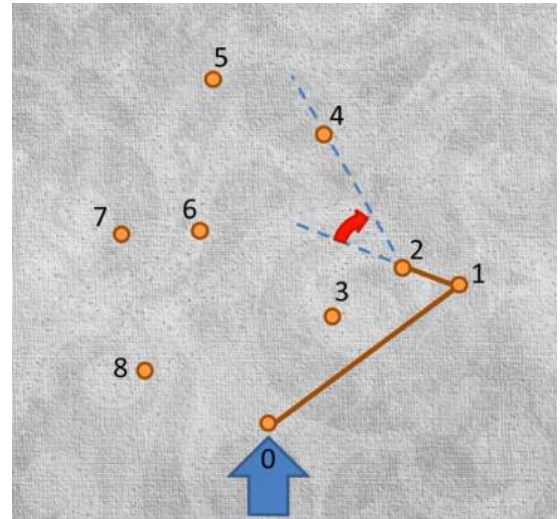


Figure 13: Another right turn: p_2 is removed. Screenshot from [3]

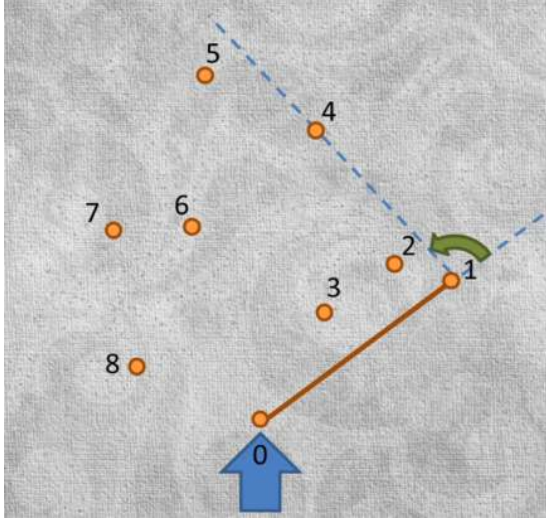


Figure 14: Orientation test for $p_0\vec{p}_1$ and $p_1\vec{p}_4$. Screenshot from [3]

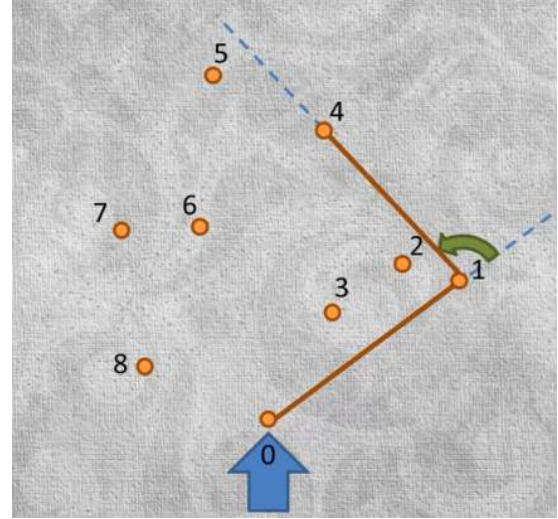


Figure 15: Valid left turn: p_4 added to stack. Screenshot from [3]

After processing all points in the sorted list, the stack contains exactly the vertices of the convex hull in counter-clockwise order, as seen in Figure 16.

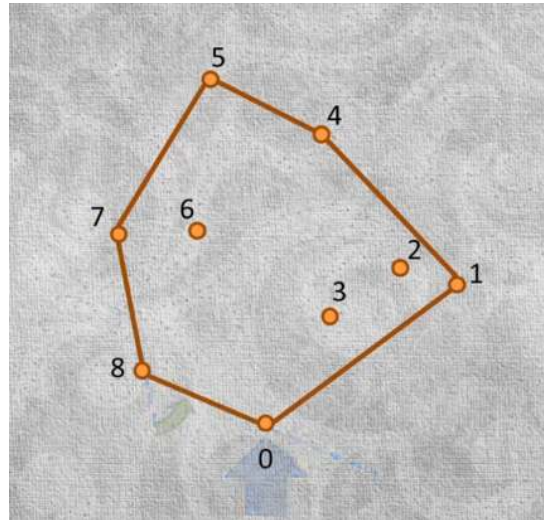


Figure 16: Final Convex Hull vertices. Screenshot from [3]

3.2.1 Pseudocode & Explanation

The Graham Scan systematically applies the orientation test. By sorting points first, the algorithm ensures we only ever move in a counter-clockwise direction. The stack serves as a temporal memory that allows the algorithm to "undo" mistakes (points that initially looked like hull vertices but were later found to be interior) in $O(1)$ time per pop.

Algorithm 2 Graham Scan Convex Hull

 GrahamScanX

Require: Set X of n distinct points**Ensure:** List L of vertices in counter-clockwise order

```

1:  $p_0 \leftarrow$  point with minimum  $y$ -coordinate (tie-break with  $x$ )
2:  $SortedPoints \leftarrow$  Sort  $X \setminus \{p_0\}$  by polar angle relative to  $p_0$ 
3:  $S \leftarrow$  Empty stack
4: Push  $p_0$  and the first two points of  $SortedPoints$  onto  $S$ 
5: for all remaining points  $p_i$  in  $SortedPoints$  do
6:   while size of  $S \geq 2$  and orientation(Next-to-top( $S$ ), Top( $S$ ),  $p_i$ )  $\leq 0$  do
7:     Pop  $S$ 
8:   end while
9:   Push  $p_i$  onto  $S$ 
10: end for
11: return  $S$  as the list  $L$ 

```

3.2.2 Time Complexity

The efficiency of the Graham Scan is a significant improvement over the brute-force approach. We can break down the total cost $T(n)$ as follows: Finding the Pivot ($O(n)$): A single pass through the n points is required to find the point with the minimum y -coordinate. Polar Sorting ($O(n \log n)$): This is the primary bottleneck of the algorithm. Sorting the $n - 1$ points by their polar angle relative to the pivot takes $O(n \log n)$ time using standard sorting algorithms like QuickSort or MergeSort. The Scan (Amortized $O(n)$): While it may appear that the nested while loop inside the for loop would lead to $O(n^2)$, we use amortized analysis. Each point is pushed onto the stack exactly once. Once a point is popped from the stack, it is never processed again. Therefore, the total number of "Pop" operations across the entire execution cannot exceed n . The total time complexity is:

$$T(n) = O(n) + O(n \log n) + O(n) = \mathbf{O(n \log n)}$$

3.2.3 python implementation

```

1
2 def get_orientation(p, q, r):
3     """ Returns orientation: >0 for CCW (left), <0 for CW (right), 0 for collinear. """
4     return (q[0] - p[0]) * (r[1] - p[1]) - (q[1] - p[1]) * (r[0] - p[0])
5
6
7 def graham_scan(points): n = len(points) if n < 3: return sorted(points)
8 # 1. Find pivot: lowest y-coordinate, then lowest x-coordinate

```

```

9 pivot = min(points, key=lambda p: (p[1], p[0]))
10
11 # 2. Sort points by polar angle relative to pivot
12 # If angles are equal, sort by distance from pivot
13 def get_angle_dist(p):
14     angle = math.atan2(p[1] - pivot[1], p[0] - pivot[0])
15     dist = (p[0] - pivot[0])**2 + (p[1] - pivot[1])**2
16     return (angle, dist)
17
18 sorted_pts = sorted([p for p in points if p != pivot], key=get_angle_dist)
19
20 # 3. Process points using a stack
21 stack = [pivot, sorted_pts[0], sorted_pts[1]]
22
23 for i in range(2, len(sorted_pts)):
24     while len(stack) >= 2 and get_orientation(stack[-2], stack[-1], sorted_pts[i]) <= 0:
25         stack.pop()
26     stack.append(sorted_pts[i])
27
28 return stack

```

3.3 Jarvis March (Gift Wrapping)

The Jarvis March algorithm, also known as the **Gift Wrapping algorithm**, is an intuitive method for computing the convex hull. Its logic is analogous to wrapping a piece of string around a set of nails. Starting from a point guaranteed to be on the hull, we "wrap" the set by identifying the next vertex that is most counter-clockwise relative to the current one.

The Jarvis March algorithm is similar to **Graham Scan**. The difference of two algorithms lies in how they find the next point. In the previous section we have seen that, Graham Scan sorts all of the points first, Figure 17 but Jarvis March finds the next point with brute-force as shown in Figures 18, 19, 20, 21 .

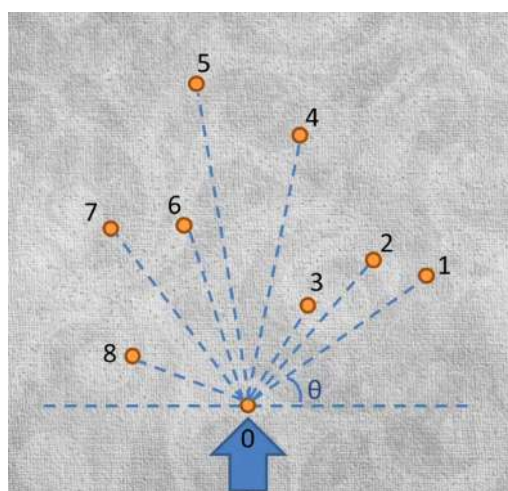


Figure 17:

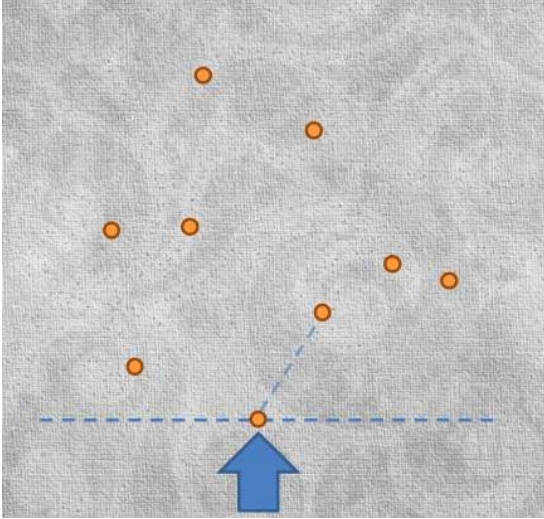


Figure 18: checking with brute-force.
Screenshot from [3]

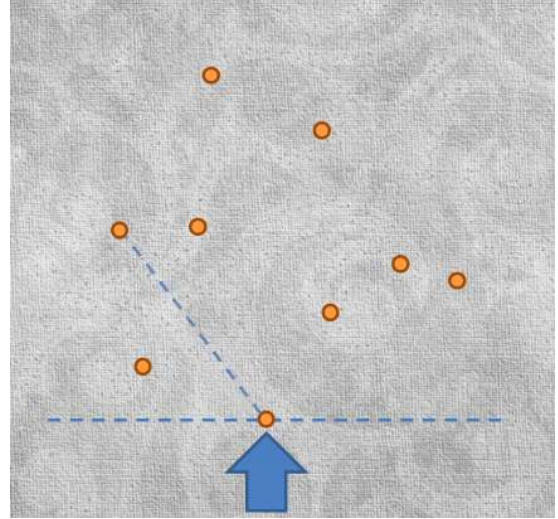


Figure 19: checking with brute-force..
Screenshot from [3]

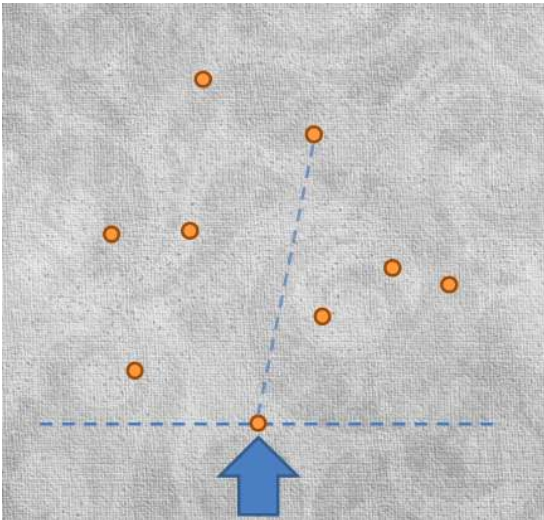


Figure 20: checking with brute-force.
Screenshot from [3]

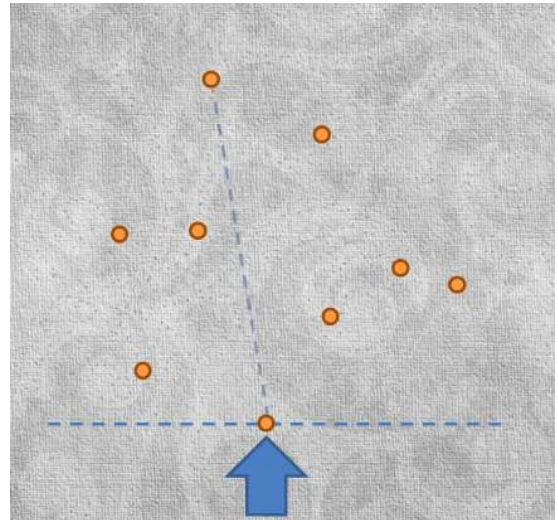


Figure 21: checking with brute-force.
Screenshot from [3]

3.3.1 Pseudocode Explanation

The algorithm begins by selecting the point with the lowest y-coordinate (the pivot p_0), which is guaranteed to be a vertex of the convex hull. From the current vertex, we search for the next vertex by testing all other points in the set. The point that is the most "counter-clockwise" (i.e., every other point lies to the left of the line formed by the current vertex and this candidate) is selected as the next vertex. This process repeats until the algorithm returns to the starting point p_0 .

Algorithm 3 Jarvis March (Gift Wrapping)

JarvisMarch X

Require: Set X of n distinct points

Ensure: List L of vertices in counter-clockwise order

```

1:  $p_{start} \leftarrow$  point in  $X$  with minimum  $y$ -coordinate
2:  $L \leftarrow [p_{start}]$ 
3:  $current \leftarrow p_{start}$ 
4: loop
5:    $next \leftarrow$  any point in  $X$  other than  $current$ 
6:   for all points  $r$  in  $X \setminus \{current, next\}$  do
7:     if orientation( $current, next, r$ )  $< 0$  then
8:        $next \leftarrow r$  {Found a point more counter-clockwise}
9:     end if
10:  end for
11:  if  $next = p_{start}$  then
12:    break {Returned to start, hull is complete}
13:  end if
14:  Add  $next$  to  $L$ 
15:   $current \leftarrow next$ 
16: end loop
17: return  $L$ 

```

3.3.2 Time Complexity

The time complexity of Jarvis March is **output-sensitive**, meaning it depends on both the number of input points (n) and the number of points that end up on the convex hull (h).

- **Outer Loop:** The algorithm runs once for each vertex on the convex hull, totaling h iterations.
- **Inner Loop:** For each hull vertex, we must scan all n points in the set to find the most counter-clockwise candidate.

Total cost is calculated as:

$$T(n) = \sum_{i=1}^h O(n) = O(nh)$$

While this is efficient if h is small (e.g., $h \log n$), in the worst-case scenario where all points are on the hull ($h=n$), the complexity degrades to $O(n^2)$.

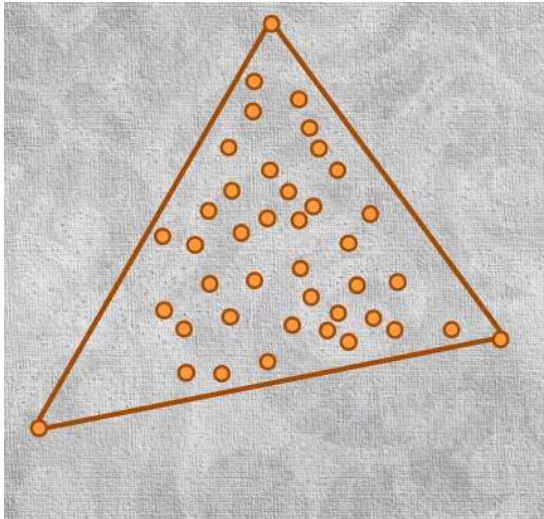


Figure 22: best case: h is the smallest. Screenshot from [3]



Figure 23: Worst case: $h=n$, all of the points are boundary of convex. Screenshot from [3]

3.3.3 python implementation

```

1
2 def get_orientation(p, q, r):
3     """Returns orientation: >0 for CCW, <0 for CW, 0 for collinear."""
4     return (q[0] - p[0]) * (r[1] - p[1]) - (q[1] - p[1]) * (r[0] - p[0])
5
6 def jarvis_march(points):
7     n = len(points) if n < 3: return sorted(points)
8
9     # 1. Start at the point with the lowest y-coordinate
10    start_point = min(points, key=lambda p: (p[1], p[0]))
11    hull = []
12    current_point = start_point
13
14    while True:
15        hull.append(current_point)
16        # Assume the first available point is the most CCW
17        next_point = points[0] if points[0] != current_point else points[1]
18
19        for r in points:
20            if r == current_point or r == next_point:
21                continue
22            # If r is more counter-clockwise than current next_point, update
23            if get_orientation(current_point, next_point, r) < 0:
24                next_point = r
25
26        current_point = next_point

```

```
27     # If we wrapped around to the beginning, we are done
28     if current_point == start_point:
29         break
30
31     return hull
```

3.4 Optimal Output-Sensitive Convex Hull Algorithms

The algorithm developed by Timothy M. Chan in 1996 [2] is a worst-case optimal, output-sensitive algorithm that combines the strengths of **Graham Scan** and **Jarvis March**. While Graham Scan is efficient at $O(n \log n)$ and Jarvis March is output-sensitive at $O(nh)$, Chan's algorithm achieves the optimal $O(n \log h)$ time complexity by using a "grouping trick" to speed up the wrapping steps. The pseudocode and detailed explanation can be found in Chan's paper [2].

4 Convex hull in practice

4.1 Convex hull in Statistics

Beyond pure geometry, the convex hull is a vital tool for robust multivariate statistics. In high-dimensional data, traditional measures like the mean are highly sensitive to outliers; however, *convex hull peeling* provides a robust alternative for location estimation. By iteratively removing the outermost layers (vertices) of a dataset, one can determine "data depth." The boundary of each hull represents a level set of the distribution, with the innermost layer yielding the Convex Hull Peeling Median (CHPM).

This geometric approach is also used for outlier detection through area sensitivity. By calculating the sensitivity $\sigma(p)$ —the reduction in hull area upon removing a point p —statisticians can identify influential anomalies. This method is particularly effective in large-scale astronomical datasets, such as the Sloan Digital Sky Survey, where it helps isolate galactic anomalies in color-magnitude diagrams without relying on rigid parametric assumptions [12] [1].

4.2 Convex hull in Computer Graphics

In computer graphics and physics engines, convex hulls are utilized to optimize real-time collision detection. While Axis-Aligned Bounding Boxes (AABB) are used in the "broad phase" to quickly discard non-colliding objects, convex hulls provide a "mid-phase" or "narrow-phase" solution. They offer a much tighter fit than bounding spheres or boxes, significantly reducing "empty space" and false positives while remaining computationally efficient for intersection tests via algorithms like Gilbert-Johnson-Keerthi (GJK) [7].

For complex, concave objects where a single hull would be inaccurate, Approximate Convex Decomposition (ACD) is employed. Algorithms like V-HACD [9] segment a mesh into a collection of convex parts, allowing simulations to maintain functional details—such as the interior of a cup or slots in a toaster. Recent advancements have even linked GJK to Frank-Wolfe optimization, enabling Nesterov acceleration to meet the high-frequency demands of haptic force-feedback systems.

4.3 Convex hull in Machine Learning Fuzzy clustering

The integration of convex hulls into machine learning has facilitated unsupervised learning models that bypass traditional distance metrics. Unlike K-means, which assumes spherical clusters, hull-based fuzzy clustering characterizes partitions as irregular convex polytopes [8]. In this framework, the membership function of a data point is determined by its proximity to the cluster's convex hull rather than a single central prototype.

This geometric membership evaluation often utilizes Gaussian or cone-shaped kernels to measure the distance from a pattern to the hull boundary. This allows for more

flexible data partitioning and is particularly effective in online sequential procedures. Such methods have proven highly efficient in complex real-world applications, ranging from medical diagnostics to the environmental monitoring of dust deposition patterns.

5 Bibliography

References

- [1] Vic Barnett. The ordering of multivariate data. *Journal of the Royal Statistical Society: Series A (General)*, 139(3):318–344, 1976.
- [2] Timothy M Chan. Optimal output-sensitive convex hull algorithms in two and three dimensions. *Discrete & computational geometry*, 16(4):361–368, 1996.
- [3] Stable Sort Youtube channel. Convex hull algorithm - graham scan and jarvis march tutorial. <https://www.youtube.com/watch?v=B2AJoSZf4M>, 2026. Screenshots from "Stable Sort" youtube channel, time: 04:00, Last retrieved: 9 Feb 2026.
- [4] Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, and Clifford Stein. *Introduction to Algorithms*. The MIT Press, Cambridge, MA, 4th edition, 2022.
- [5] Jöran Friberg. Methods and traditions of babylonian mathematics. *Historia Mathematica*, 8(3):277–318, 1981.
- [6] Ronald L Graham and F Frances Yao. Finding the convex hull of a simple polygon. *Journal of Algorithms*, 4(4):324–331, 1983.
- [7] Ming C Lin and Dinesh Manocha. Collision detection. In *Handbook of Data Structures and Applications*, pages 889–902. Chapman and Hall/CRC, 2018.
- [8] Luca Liparulo, Andrea Proietti, and Massimo Panella. Fuzzy clustering using the convex hull as geometrical model. *Advances in Fuzzy Systems*, 2015(1):265135, 2015.
- [9] Khaled Mamou, E Lengyel, and A Peters. Volumetric hierarchical approximate convex decomposition. *Game engine gems*, 3:141–158, 2016.
- [10] Otto Neugebauer. Egyptian mathematics and astronomy. In *The Exact Sciences in Antiquity*, pages 71–96. Dover Publications, 2 edition, 1969. Originally published 1957.
- [11] Vincenzo De Risi, editor. *Mathematizing Space: The Objects of Geometry from Antiquity to the Early Modern Age*. Trends in the History of Science. Birkhäuser / Springer International Publishing, Cham, Switzerland, 2015.
- [12] Guillaume Staerman, Pavlo Mozharovskyi, Stéphan Clémen, et al. The area of the convex hull of sampled curves: a robust functional statistical depth measure. In *International Conference on Artificial Intelligence and Statistics*, pages 570–579. PMLR, 2020.

- [13] TutorialsPoint. C++ program to implement graham scan algorithm to find the convex hull, 2025.
- [14] Wikipedia contributors. Convex hull. https://en.wikipedia.org/wiki/Convex_hull, 2026. Wikipedia, The Free Encyclopedia. Last retrieved: 9 Feb 2026.
- [15] Wikipedia contributors. Euclidean geometry. https://en.wikipedia.org/wiki/Euclidean_geometry, 2026. Wikipedia, The Free Encyclopedia. Last retrieved: 9 Feb 2026.
- [16] Wikipedia contributors. Geometry. <https://en.wikipedia.org/wiki/Geometry>, 2026. Wikipedia, The Free Encyclopedia. Last retrieved: 9 Feb 2026.